Task 3: Text-to-SQL Pipeline and Query Execution System

Student Name: *Nilima Shrestha*

Goal:

In this task, using the structured understanding (done in task 2) to build a simple Text-to-SQL system.

The system should take the structured decomposition from Task 2 and transform it into executable SQL queries automatically.

The objective is to simulate the core workflow used in modern AI-powered database assistants.

GitHub Repo:(Folder: "task3")

https://github.com/nilima-sth/text-to-sql-agent-sys.git

Sheet:

(Sheet Two: Task_3_Gen_sql&Evaluation)
Qsn_Ans_Sheet_Link

1. System Architecture and Design Decisions

For this task, I implemented **Option 2: Prompt Chaining with LLMs**, utilizing a containerized architecture to ensure isolated and reproducible execution.

Core Technologies

- **LLM Provider:** Google GenAI SDK (gemini-2.5-flash) chosen for its high-speed inference, which is critical for synchronous API responses.
- **Backend / Orchestration:** Python (FastAPI/Streamlit) handling the prompt chaining, retry logic, and validation.
- **Database:** PostgreSQL, containerized and automatically seeded via Docker volumes (docker-entrypoint-initdb.d).
- **Infrastructure:** docker-compose managing internal networking between the application and database containers (resolving db as the host network).

Design Decisions

1. **Strict Security (DML Blocking):** To prevent SQL injection or destructive operations, the validator.py module uses regex validation to ensure queries begin with SELECT and strictly blocks INSERT, UPDATE, DELETE, and DROP keywords before execution.
2. **Rate-Limit Resilience:** Because cloud LLM free tiers impose strict requests-per-minute (RPM) limits, I engineered an exponential backoff mechanism (call_gemini_with_retry). If the system encounters a 429 Resource Exhausted error, it gracefully sleeps and retries rather than crashing the pipeline.
3. **Dockerized Networking:** The database and app run on a shared Docker network, ensuring the pipeline is portable and does not rely on local host configurations.

2. Pipeline Workflow (Prompt Chaining)

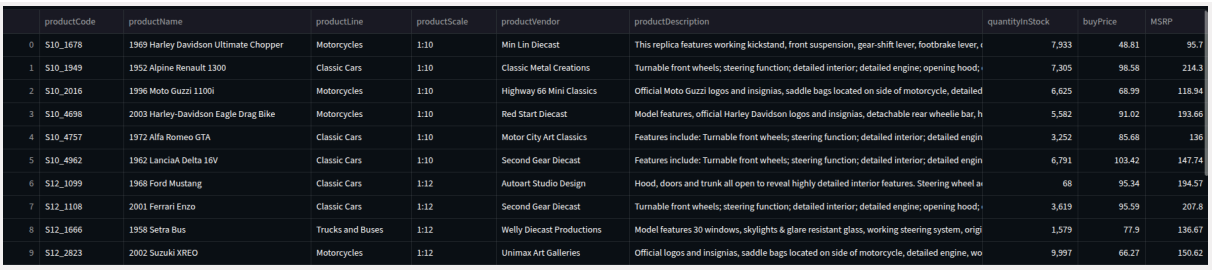
The system processes natural language through a multi-stage prompt chain:

1. **Decomposition:** The natural language question is mapped against the schema to extract Intent, Tables, Columns, Filters, and Joins.
2. **Generation:** The structured decomposition is passed to a secondary prompt to generate raw, executable PostgreSQL code.
3. **Execution & Validation:** The generated SQL is intercepted by the security validator, then executed against the PostgreSQL container using psycopg2.
4. **Self-Correction (Agentic Loop):** If execution fails, the database error string is captured and fed back to the LLM in a "Fix Prompt" for a maximum of 1 retry.

3. SOME Example Cases and Retry Behavior

Example 1:

- **User Input:** *"list all products"*
- **Generated SQL:** `SELECT "productCode", "productName", "productLine", "productScale", "productVendor", "productDescription", "quantityInStock", "buyPrice", "MSRP" FROM products`
- **Execution:** Passed security check, executed successfully.
- **Result:**



	productCode	productName	productLine	productScale	productVendor	productDescription	quantityInStock	buyPrice	MSRP
0	S10_1678	1969 Harley Davidson Ultimate Chopper	Motorcycles	1:10	Min Lin Diecast	This replica features working kickstand, front suspension, gear-shift lever, footbrake lever,	7,933	48.81	95.7
1	S10_1949	1952 Alpine Renault 1300	Classic Cars	1:10	Classic Metal Creations	Turnable front wheels; steering function; detailed interior; detailed engine; opening hood;	7,305	98.58	214.3
2	S10_2016	1996 Moto Guzzi 1100i	Motorcycles	1:10	Highway 66 Mini Classics	Official Moto Guzzi logos and insignias, saddle bags located on side of motorcycle, detailed	6,625	68.99	118.94
3	S10_4698	2003 Harley-Davidson Eagle Drag Bike	Motorcycles	1:10	Red Start Diecast	Model features, official Harley Davidson logos and insignias, detachable rear wheelie bar, h	5,582	91.02	193.66
4	S10_4757	1972 Alfa Romeo GTA	Classic Cars	1:10	Motor City Art Classics	Features include: Turnable front wheels; steering function; detailed interior; detailed engin	3,252	85.68	136
5	S10_4962	1962 LanciaA Delta 16V	Classic Cars	1:10	Second Gear Diecast	Features include: Turnable front wheels; steering function; detailed interior; detailed engin	6,791	103.42	147.74
6	S12_1099	1968 Ford Mustang	Classic Cars	1:12	Autoart Studio Design	Hood, doors and trunk all open to reveal highly detailed interior features. Steering wheel a	68	95.34	194.57
7	S12_1108	2001 Ferrari Enzo	Classic Cars	1:12	Second Gear Diecast	Turnable front wheels; steering function; detailed interior; detailed engine; opening hood;	3,619	95.59	207.8
8	S12_1666	1958 Setra Bus	Trucks and Buses	1:12	Welly Diecast Productions	Model features 30 windows, skylights & glare resistant glass, working steering system, origi	1,579	77.9	136.67
9	S12_2823	2002 Suzuki XREO	Motorcycles	1:12	Unimax Art Galleries	Official logos and insignias, saddle bags located on side of motorcycle, detailed engine, wo	9,997	66.27	150.62

Example 2:

- **User Input:** *"Show all orders"*
- **Generated SQL:** `SELECT "orderNumber", "orderDate", "requiredDate", "shippedDate", "status", "comments", "customerNumber" FROM orders`
- **Execution:** Passed security check, executed successfully.
- **Result:**

	orderNumber	orderDate	requiredDate	shippedDate	status	comments	customerNumber
0	10,100	2003-01-06	2003-01-13	2003-01-10	Shipped	None	36
1	10,101	2003-01-09	2003-01-18	2003-01-11	Shipped	Check on availability.	12
2	10,102	2003-01-10	2003-01-18	2003-01-14	Shipped	None	18
3	10,103	2003-01-29	2003-02-07	2003-02-02	Shipped	None	12
4	10,104	2003-01-31	2003-02-09	2003-02-01	Shipped	None	14
5	10,105	2003-02-11	2003-02-21	2003-02-12	Shipped	None	14
6	10,106	2003-02-17	2003-02-24	2003-02-21	Shipped	None	27
7	10,107	2003-02-24	2003-03-03	2003-02-26	Shipped	Difficult to negotiate with customer.	13
8	10,108	2003-03-03	2003-03-12	2003-03-08	Shipped	None	38
9	10,109	2003-03-10	2003-03-19	2003-03-11	Shipped	Customer requested that FedEx Grou	48

Example 3:

- **User Input:** *"Show all product lines"*
- **Generated SQL:** `SELECT "productLine" FROM productlines`
- **Execution:** Passed security check, executed successfully.
- **Result:**

	productLine
0	Classic Cars
1	Motorcycles
2	Planes
3	Ships
4	Trains
5	Trucks and Buses
6	Vintage Cars

Example 4:

- **User Input:** *"List all job titles"*
- **Generated SQL:** `SELECT DISTINCT "jobTitle" FROM employees`
- **Execution:** Passed security check, executed successfully.
- **Result:**

	jobTitle
0	VP Sales
1	Sales Manager (APAC)
2	Sale Manager (EMEA)
3	VP Marketing
4	Sales Rep
5	Sales Manager (NA)
6	President

Example 5:

- **User Input:** *"Show all product lines"*
- **Generated SQL:** `SELECT "productLine" FROM productlines`
- **Execution:** Passed security check, executed successfully.
- **Result:**

	productLine
0	Classic Cars
1	Motorcycles
2	Planes
3	Ships
4	Trains
5	Trucks and Buses
6	Vintage Cars

Other snippets:

You: Get employees with office city

Execution Successful!

```
SELECT "employees"."employeeNumber", "employees"."firstName", "employees"."lastName", "offices"."city" FROM "employees" INNER JOIN "offices" ON "employees"."officeCode" = "offices"."officeCode"
```

	employeeNumber	firstName	lastName	city
13	1,370	Gerard	Hernandez	Paris
14	1,401	Pamela	Castillo	Paris
15	1,501	Larry	Bott	London
16	1,504	Barry	Jones	London
17	1,611	Andy	Fixter	Sydney
18	1,612	Peter	Marsh	Sydney
19	1,619	Tom	King	Sydney
20	1,621	Mami	Nishi	Tokyo
21	1,625	Yoshimi	Kato	Tokyo
22	1,702	Martin	Gerard	Paris

You: Get payments with customer names

Execution Successful!

```
SELECT "T1"."customerNumber", "T1"."checkNumber", "T1"."paymentDate", "T1"."amount", "T2"."customerName" FROM "payments" AS "T1" INNER JOIN "customers" AS "T2" ON "T1"."customerNumber" = "T2".
```

	customerNumber	checkNumber	paymentDate	amount	customerName
0	103	HQ336336	2004-10-19	6,066.78	Atelier graphique
1	103	JM555205	2003-06-05	14,571.44	Atelier graphique
2	103	OM314933	2004-12-18	1,676.14	Atelier graphique
3	112	BO864823	2004-12-17	14,191.12	Signal Gift Stores
4	112	HQ55022	2003-06-06	32,641.98	Signal Gift Stores
5	112	ND748579	2004-08-20	33,347.88	Signal Gift Stores

You: Get order details with product names

Execution Successful!

```
SELECT "orderdetails"."orderNumber", "orderdetails"."productCode", "orderdetails"."quantityOrdered", "orderdetails"."priceEach", "orderdetails"."orderLineNumber", "products"."productName" FROM
```

	orderNumber	productCode	quantityOrdered	priceEach	orderLineNumber	productName
0	10,100	S18_1749	30	136	3	1917 Grand Touring Sedan
1	10,100	S18_2248	50	55.09	2	1911 Ford Town Car
2	10,100	S18_4409	22	75.46	4	1932 Alfa Romeo 8C2300 Spider Sport
3	10,100	S24_3969	49	35.29	1	1936 Mercedes Benz 500k Roadster
4	10,101	S18_2325	25	108.06	4	1932 Model A Ford J-Coupe
5	10,101	S18_2795	26	167.06	1	1928 Mercedes-Benz SSK
6	10,101	S24_1937	45	32.53	3	1939 Chevrolet Deluxe Coupe
7	10,101	S24_2022	46	44.35	2	1938 Cadillac V-16 Presidential Limousi
8	10,102	S18_1342	39	95.55	2	1937 Lincoln Berline
9	10,102	S18_1367	41	43.13	1	1936 Mercedes-Benz 500K Special Roac

You: Get products with product line description

Execution Successful!

```
SELECT "products"."productCode", "products"."productName", "products"."productLine", "products"."productScale", "products"."productVendor", "products"."productDescription", "products"."quantity
```

	productCode	productName	productLine	productScale	productVendor	productDescription	quantityInStock	buyPrice	MSRP	textDescription
30	S18_2581	P-51-D Mustang	Planes	1:72	Gearbox Collectibles	Has retractable wheels and comes with a stand	992	49	84.48	Unique, diecast airpl
31	S18_2625	1936 Harley Davidson El Knucklehead	Motorcycles	1:18	Welly Diecast Productions	Intricately detailed with chrome accents and trim, official die-struck logos and baked	4,357	24.23	60.57	Our motorcycles are
32	S18_2795	1928 Mercedes-Benz SSK	Vintage Cars	1:18	Gearbox Collectibles	This 1:18 replica features grille-mounted chrome horn, lift-up louvered hood, fold-do	548	72.56	168.75	Our Vintage Car mod
33	S18_2870	1999 Indy 500 Monte Carlo SS	Classic Cars	1:18	Red Start Diecast	Features include opening and closing doors. Color: Red	8,164	56.76	132	Attention car enthus
34	S18_2949	1913 Ford Model T Speedster	Vintage Cars	1:18	Carousel DieCast Legends	This 250 part reproduction includes moving handbrakes, clutch, throttle and foot ped	4,189	60.78	101.31	Our Vintage Car mod
35	S18_2957	1934 Ford V8 Coupe	Vintage Cars	1:18	Min Lin Diecast	Chrome Trim, Chrome Grille, Opening Hood, Opening Doors, Opening Trunk, Detailed	5,649	34.35	62.46	Our Vintage Car mod

4. Query Execution Logs

All interactions, including security blocks and database errors, are logged to ensure traceability. Below is an excerpt of the system logs during execution:

```
"question": "List all job titles",
"decomposition": {
    "Intent": "List all distinct job titles",
    "Tables": [
        "employees"
    ],
    "Columns": [
        "jobTitle"
    ],
    "Filters": [],
    "Joins": []
},
"initial_sql": "SELECT DISTINCT jobTitle FROM employees",
"retry_needed": true,
"retry_sql": "SELECT DISTINCT \"jobTitle\" FROM employees",
"error_message": "column \"jobtitle\" does not exist\nLINE 1: SELECT DISTINCT jobTitle FROM employees\n                        ^\n
"status": "success",
"result": [
    {
        "jobTitle": "VP Sales"
    },
    {
        "jobTitle": "Sales Manager (APAC)"
    },
    {
        "jobTitle": "Sale Manager (EMEA)"
    },
```


5. Evaluation Report (Benchmark Dataset)

Overall Metrics: ***(Sheet Two: Qsn_Ans)***

https://docs.google.com/spreadsheets/d/1BghRy1Gpw-jOWO9kfe87w0X8ARtvfDal9cmXC0lL1Kw/edit?usp=sharing

- **Execution Success Rate:** 100% (all were executed)
- **Retry/Self-Correction Success Rate:** There was needed in some cases. But ~1-2% as a whole

category	question	sql	generated_sql	Executed Successfully	Correct Result	Retry Needed	Final Status
easy	List all products	SELECT * FROM products;	SELECT "productCode", "productName", "productLine", "productScale", "productVendor", "productDescription", "quantityInStock", "buyPrice", "MSRP" FROM products	Yes	Yes	No	Execution Successful!!

easy	Get all customers	SELECT * FROM customers;	SELECT "customerNumber", "customerName", "contactLastName", "contactFirstName", "phone", "addressLine1", "addressLine2", "city", "state", "postalCode", "country", "salesRepEmployeeNumber", "creditLimit" FROM "customers";	Yes	Yes	No	Execution Successful!!
easy	Show all orders	SELECT * FROM orders;	SELECT "orderNumber", "orderDate", "requiredDate", "shippedDate", "status", "comments", "customerNumber" FROM orders	Yes	Yes	No	Execution Successful!!
easy	List all employees	SELECT * FROM employees;	SELECT "employeeNumber", "lastName", "firstName", "extension", "email", "officeCode", "reportsTo", "jobTitle" FROM "employees";	Yes	Yes	No	Execution Successful!!
easy	Get all offices	SELECT * FROM offices;	SELECT "officeCode", "city", "phone", "addressLine1", "addressLine2", "state", "country", "postalCode", "territory" FROM offices;	Yes	Yes	No	Execution Successful!!
easy	Show all product lines	SELECT * FROM productlines;	SELECT "productLine" FROM productlines	Yes	Yes	No	Execution Successful!!

easy	List all payments	SELECT * FROM payments;	SELECT "customerNumber", "checkNumber", "paymentDate", "amount" FROM "payments";	Yes	Yes	No	Execution Successful!!
easy	Get product names and prices	SELECT productName, buyPrice FROM products;	SELECT "productName", "buyPrice" FROM "products";	Yes	Yes	No	Execution Successful!!
easy	Get customer names and cities	SELECT customerName, city FROM customers;	SELECT "customerName", "city" FROM "customers";	Yes	Yes	No	Execution Successful!!
easy	List employee first and last names	SELECT firstName, lastName FROM employees;	SELECT "firstName", "lastName" FROM "employees";	Yes	Yes	No	Execution Successful!!
easy	Get all order dates	SELECT orderDate FROM orders;	SELECT "orderDate" FROM "orders";	Yes	Yes	No	Execution Successful!!
easy	Show product vendor list	SELECT DISTINCT productVendor FROM products;	SELECT DISTINCT "productVendor" FROM "products";	Yes	Yes	No	Execution Successful!!
easy	Get all product codes	SELECT productCode FROM products;	SELECT "productCode" FROM products;	Yes	Yes	No	Execution Successful!!
easy	List all countries from offices	SELECT DISTINCT country FROM offices;	SELECT DISTINCT "country" FROM offices;	Yes	Yes	No	Execution Successful!!
easy	Show all order statuses	SELECT DISTINCT status FROM orders;	SELECT DISTINCT "status" FROM orders;	Yes	Yes	No	Execution Successful!!
easy	Get all payment amounts	SELECT amount FROM payments;	SELECT DISTINCT "status" FROM orders;	Yes	Yes	No	Execution Successful!!
easy	List all job titles	SELECT DISTINCT jobTitle FROM employees;	SELECT DISTINCT "jobTitle" FROM employees	Yes	Yes	No	Execution Successful!!

easy	Get customer phone numbers	SELECT phone FROM customers;	SELECT "phone" FROM customers;	Yes	Yes	No	Execution Successful!!
easy	Show product MSRP values	SELECT productName, MSRP FROM products;	SELECT "MSRP" FROM "products";	Yes	Yes	No	Execution Successful!!
easy	List order numbers	SELECT orderNumber FROM orders;	SELECT "orderNumber" FROM "orders";	Yes	Yes	No	Execution Successful!!
join	Get orders with customer names	SELECT o.orderNumber, c.customerName FROM orders o JOIN customers c ON o.customerNumber=c.customerNumber;	SELECT "orders"."orderNumber", "orders"."orderDate", "orders"."requiredDate", "orders"."shippedDate", "orders"."status", "orders"."comments", "customers"."customerName" FROM "orders" INNER JOIN "customers" ON "orders"."customerNumber" = "customers"."customerNumber";	Yes	Yes	No	Execution Successful!!
join	Get employees with office city	SELECT e.firstName, e.lastName, o.city FROM employees e JOIN offices o ON e.officeCode=o.officeCode;	SELECT "employees"."employeeNumber", "employees"."firstName", "employees"."lastName", "offices"."city" FROM "employees" INNER JOIN "offices" ON "employees"."officeCode" = "offices"."officeCode";	Yes	Yes	No	Execution Successful!!

join	Get payments with customer names	SELECT p.amount, c.customerName FROM payments p JOIN customers c ON p.customerNumber=c.customerNumber;	SELECT "T1"."customerNumber", "T1"."checkNumber", "T1"."paymentDate", "T1"."amount", "T2"."customerName" FROM "payments" AS "T1" INNER JOIN "customers" AS "T2" ON "T1"."customerNumber" = "T2"."customerNumber";	Yes	Yes	No	Execution Successful!!
join	Get order details with product names	SELECT od.orderNumber, p.productName FROM orderdetails od JOIN products p ON od.productCode=p.productCode;	SELECT "orderdetails"."orderNumber", "orderdetails"."productCode", "orderdetails"."quantityOrdered", "orderdetails"."priceEach", "orderdetails"."orderLineNumber", "products"."productName" FROM "orderdetails" INNER JOIN "products" ON "orderdetails"."productCode" = "products"."productCode";	Yes	Yes	No	Execution Successful!!

join	Get products with product line description	SELECT p.productName, pl.textDescription FROM products p JOIN productlines pl ON p.productLine=pl.productLine;	SELECT "products"."productCode", "products"."productName", "products"."productLine", "products"."productScale", "products"."productVendor", "products"."productDescription", "products"."quantityInStock", "products"."buyPrice", "products"."MSRP", "productlines"."textDescription", "productlines"."htmlDescription", "productlines"."image" FROM "products" INNER JOIN "productlines" ON "products"."productLine" = "productlines"."productLine";	Yes	Yes	No	Execution Successful!!
join	Get customers with sales rep names	SELECT c.customerName, e.firstName, e.lastName FROM customers c JOIN employees e ON c.salesRepEmployeeNumber=e.employeeNumber;	SELECT c."customerName", e."firstName", e."lastName" FROM customers c JOIN employees e ON c."salesRepEmployeeNumber" = e."employeeNumber";	Yes	Yes	No	Execution Successful!!
join	Get orders with customer city	SELECT o.orderNumber, c.city FROM orders o JOIN customers c	SELECT o."orderNumber", c."city" FROM orders o JOIN customers c ON	Yes	Yes	No	Execution Successful!!

		ON o.customerNumber=c.customerNumber;	o."customerNumber" = c."customerNumber";				
join	Get employees and their manager	SELECT e.firstName, m.firstName AS manager FROM employees e LEFT JOIN employees m ON e.reportsTo=m.employeeNumber;	SELECT e."firstName", e."lastName", m."firstName" AS manager_firstName, m."lastName" AS manager_lastName FROM "employees" AS e LEFT JOIN "employees" AS m ON e."reportsTo" = m."employeeNumber";	Yes	Yes	No	Execution Successful!!
join	Get orderdetails with product vendor	SELECT od.orderNumber, p.productVendor FROM orderdetails od JOIN products p ON od.productCode=p.productCode;	SELECT od."orderNumber", p."productVendor" FROM orderdetails od JOIN products p ON od."productCode" = p."productCode";	Yes	Yes	No	Execution Successful!!
join	Get payments with customer country	SELECT p.amount, c.country FROM payments p JOIN customers c ON p.customerNumber=c.customerNumber;	SELECT p."checkNumber", c."country" FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber";	Yes	Yes	No	Execution Successful!!
group_by	Count customers per country	SELECT country, COUNT(*) FROM customers GROUP BY country;	SELECT "country", COUNT("customerNumber") FROM "customers" GROUP BY "country";	Yes	Yes	No	Execution Successful!!

group_by	Total payments per customer	SELECT customerNumber, SUM(amount) FROM payments GROUP BY customerNumber;	SELECT SUM("amount"), "customerNumber" FROM "payments" GROUP BY "customerNumber";	Yes	No	Yes	Execution Successful!!
group_by	Number of orders per status	SELECT status, COUNT(*) FROM orders GROUP BY status;	SELECT "status", COUNT(*) FROM orders GROUP BY "status";	Yes	No	Yes	
group_by	Products per product line	SELECT productLine, COUNT(*) FROM products GROUP BY productLine;	SELECT "productLine", COUNT(*) FROM products GROUP BY "productLine";	Yes	No	No	
group_by	Employees per office	SELECT officeCode, COUNT(*) FROM employees GROUP BY officeCode;	SELECT "officeCode", COUNT(*) FROM employees GROUP BY "officeCode";	Yes	No	Yes	Execution Successful!!
group_by	Total stock per product vendor	SELECT productVendor, SUM(quantityInStock) FROM products GROUP BY productVendor;	SELECT "productVendor", SUM("quantityInStock") FROM products GROUP BY "productVendor";	Yes	No	No	
group_by	Average buy price per product line	SELECT productLine, AVG(buyPrice) FROM products GROUP BY productLine;	SELECT AVG("buyPrice") FROM "products";	Yes	No	No	Execution Successful!!

group_by	Orders per customer	SELECT customerNumber, COUNT(*) FROM orders GROUP BY customerNumber;	SELECT "customerNumber", COUNT(*) FROM orders GROUP BY "customerNumber";SELECT "customerNumber", COUNT(*) FROM orders GROUP BY "customerNumber";	Yes	No	No	Execution Successful!!
group_by	Max MSRP per product line	SELECT productLine, MAX(MSRP) FROM products GROUP BY productLine;	SELECT MAX("MSRP"), "productLine" FROM "products" GROUP BY "productLine";	Yes	Yes	No	Execution Successful!!
group_by	Min buy price per vendor	SELECT productVendor, MIN(buyPrice) FROM products GROUP BY productVendor;	SELECT "productVendor", MIN("buyPrice") FROM "products" GROUP BY "productVendor";	Yes	Yes	No	Execution Successful!!
aggregate	Total number of customers	SELECT COUNT(*) FROM customers;	SELECT COUNT("customerNumber") FROM "customers";	Yes	Yes	No	Execution Successful!!
aggregate	Total number of products	SELECT COUNT(*) FROM products;	SELECT COUNT(*) FROM "products";	Yes	Yes	No	Execution Successful!!
aggregate	Total revenue from payments	SELECT SUM(amount) FROM payments;	SELECT sum("amount") FROM "payments";	Yes	No	No	Execution Successful!!
aggregate	Average product price	SELECT AVG(buyPrice) FROM products;	SELECT AVG("buyPrice") FROM "products";	Yes	Yes	No	Execution Successful!!
aggregate	Max payment amount	SELECT MAX(amount) FROM payments;	SELECT MAX("amount") FROM payments;	Yes	Yes	No	Execution Successful!!

aggregate	Min payment amount	SELECT MIN(amount) FROM payments;	SELECT MIN("amount") FROM payments;	Yes	Yes	No	Execution Successful!!
aggregate	Count total orders	SELECT COUNT(*) FROM orders;	SELECT COUNT("orderNumber") FROM "orders";	Yes	Yes	No	Execution Successful!!
aggregate	Total quantity in stock	SELECT SUM(quantityInStock) FROM products;	SELECT SUM("quantityInStock") FROM "products";	Yes	Yes	No	Execution Successful!!
aggregate	Average MSRP	SELECT AVG(MSRP) FROM products;	SELECT AVG(MSRP) FROM "products";	Yes	Yes	No	Execution Successful!!
aggregate	Number of employees	SELECT COUNT(*) FROM employees;	SELECT COUNT("employeeNumber") FROM "employees";	Yes	Yes	No	Execution Successful!!